

o_que_e_angular

O que é Framework Angular ↗

1. INDEX

1. [Resumo do conteúdo](#)

2. **Introdução**

1. [Objetivo.](#)
2. [Pre-requisitos.](#)
3. [Benefícios.](#)
4. [Desvantagens.](#)

3. **Conteúdo estudado.**

1. [Instalar](#)
2. [Como criar um novo projeto Angular](#)
3. [Como criar um componente usando o angular/cli](#)
4. [Como criar biblioteca com o Angular 2](#)
5. [Como gerar projeto para publicar na web](#)
6. [Como criar módulos](#)
7. [Como usar rotas em Angular 2](#)
 1. [Arquivos para implementação de Rotas](#)
 2. [Exibindo as páginas do projeto](#)
 3. [Rotas filhas](#)
 4. [Guardas de Rota \(Prevenir o acesso não autorizado\).](#)
8. [Assunto 08](#)
9. [Assunto 09](#)
10. [Assunto 10](#)

4. Exemplos de uso do angular:

1. [Como criar formulário de login no angular](#)
2. [Ocultar a barra de menu quando o usuário não tiver autenticado](#)
3. [Usando guarda de rotas para usuário não autenticado](#)

5. **Referências globais.**

6. **Histórico.**

2. CONTEÚDO

1. **Resumo do conteúdo:**

1. Descreve um resumo de como foi feito esse documento com as facilidade encontradas para produzi-lo e dificuldade encontrada.

2. **Introdução**

1. Objetivo:

1. [Angular](#) é um framework escrito em [typescript](#) criado pelo Google para criação de aplicativos clientes web onde o mesmo se comporta muito parecido com aplicativo desktop ou seja: aplicação single page (Página única).
2. O Angular tem licença MIT e além do Google existe uma comunidade atuante que melhoram o produto todos os dias saindo uma nova versão a cada 6 meses.
3. Outra característica que considero importante é que cada versão será compatível com a anterior.
4. [Introdução aos conceitos do framework angular 2.](#)
5. [Veja o que é Angular?](#)
6. [ BACK]

2. Pre-requisitos:

1. Programas necessários devem estar instalados:
 1. [nodejs](#)
 2. [npm](#)
 3. [vscode](#)
 4. [typescript](#)
2. Conhecimento das linguagens:
 1. [html](#)
 2. [css](#)
 3. [typescript](#)
 4. [javascript](#)
 5. [sql](#)
 6. [Conhecimento do conceito de programação orientada a objetos.](#)
3. [ BACK]

3. Benefícios:

1. O framework Angular ajuda na criação de Single-Page Applications, com nível de produtividade e qualidade acima da média é open source e tem uma comunidade muito ativa.
2. O angular por ser framework possui tudo que é preciso para construção de aplicativos web.
3. Por ser escrito em typescript é estável e permite construção de aplicativos sem erros de sintaxe em tempo de execução como ocorre com javascript.
4. O Angular adapta e estende o HTML tradicional para uma experiência otimizada, com conteúdo dinâmico e ligação direta dos dados, conhecida

como [two-way data-binding \(ligação de dados bidirecional\)](#), que abre a possibilidade para sincronização automática de modelos e visualizações.

5. [← BACK]

4. Desvantagens.

1. A desvantagens é relativa ou seja: Caso seja um programador que não tenha conhecimento de programação orientada a objetos terá dificuldade de entender porque o Angular é todo orientado a objetos.
2. Para programadores que conhecem javascript vai estranhar a linguagem typescript porque a mesma permite identificar erros de sintaxe em tempo de edição do código.

3. [← BACK]

3. Conteúdo estudado

1. Instalar o angular

1. Para instalar a versão mais recente do Node.js, vá para <https://nodejs.org> e baixe o instalador para sua plataforma e instale-o. Isso instalará o Node.js e o NPM em sua máquina local.
2. Instalar Angular no linux

1. Código ShellScript

```
# checar versão do node instalada.
node -v

# Checar a ultima versão no npm
npm -v

# Instalar a ultima versão do npm caso use um
# espaço de trabalho para cada projeto
sudo npm install -g npm

# Caso existe um espaço de trabalho para vários
# projetos o ideal é instalar o pacote https://pnpm.io/
sudo npm install -g pnpm

# O Angular oferece muitas bibliotecas e pacotes
# para o desenvolvimento de aplicativos.
# O Angular CLI também é usado para gerar,
# construir, executar e implantar aplicativos Angular.
sudo npm install -g @angular/cli

# Para visualizar a versão atual do angular
ng --version
```

3. Referências:

1. Angular CLI: Estrutura do projeto

4. [←BACK]

2. Como criar um novo projeto Angular

1. A instalação do pacote **@angular/cli** adiciona no prompt do shellscript o comando **ng** onde mesmo possui comando **new** que é usado para criar um espaço de trabalho completo (todos os componentes necessários) para angular criar um novo projeto. [Veja mais...](#)

2. Passo a passo.

1. Abre o console do linux **Crt+alt+T** e selecione a pasta raiz de todos os projetos angular 2.
2. Execute os comandos abaixo para criar e executar o novo projeto:

```
# Para criar um novo projeto
ng new nome-do-projeto --routing

# Perguntas que o comando ng new nome-do-projeto
faz:
  ? Você gostaria de adicionar o roteamento
angular? Y/N
  ? Qual formato de folha de estilo você gostaria
de usar? (Use as setas)
    > CSS
      SCSS   [ https://sass-
lang.com/documentation/syntax#scss ]
      Sass   [ https://sass-
lang.com/documentation/syntax#the-indented-syntax ]
      Less   [ http://lesscss.org

# Move apar a pasta do projeto criado
cd ./nome-do-projeto

# Executa a aplicação criada pelo comando: .
ng serve
```

1. NOTAS:

1. Em seu navegador, abra <http://localhost:4200> para ver o novo aplicativo em execução. Quando você usa o comando **ng serve** para construir um aplicativo e atendê-lo localmente, o servidor reconstrói automaticamente o aplicativo e recarrega a página quando você altera qualquer um dos arquivos de origem.
2. O comando **ng new nome-do-projeto** cria os [arquivos de configurações](#) do espaço de trabalho. O arquivo

[readme.md](#) informa tudo que precisa fazer para executar o projeto criado.

3. [Vídeo aula sobre criação de novo projeto com a cli](#)
4. Pastas criadas pelo comando **ng new nome-do-projeto**:

1. **src/**

1. Pasta com todos os arquivos do projeto.

2. **src/app/**

1. Pasta para os componentes do projeto.

3. **src/assets/**

1. Cada build configuração de destino pode incluir uma assets matriz que lista os arquivos ou pastas que você deseja copiar no estado em que se encontram ao construir seu projeto. Por padrão, a **src/assets/** pasta e **src/favicon.ico** são copiados. [Veja mais....](#)

4. **src/environments/**

1. Pasta de um projeto que contém o arquivo de configuração base **environment.ts**, que fornece um ambiente padrão. Você pode adicionar padrões de substituição para ambientes adicionais, como produção e teste, em arquivos de configuração específicos de destino. [Veja mais...](#)

5. **node_modules/**

1. Pasta com as dependência do Angular para o projeto.

5. Arquivos criadas pelo comando **ng new nome-do-projeto**:

1. **./src/app/app.component.css**

1. Este arquivo é usado para registrar todo o código **.css** da aplicação. [Veja mais...](#)

2. **./src/app/app.component.html**

1. Arquivo **.html** para o template principal do projeto. [Veja a sintaxe do modelo...](#)
2. [templates-and-views](#)

3. **./src/app/app.component.ts**

1. Arquivo typescript de componente contém a classe com o código usado pelo template

`./src/app/app.component.html`

2. [Veja a documentação criando componente](#)

4. **`./src/app/app.component.spec.ts`**

1. Este arquivo é usado para fazer teste unitário da classe.
2. Mais informações [clique aqui...](#)

5. **`src/styles.css`**

1. Arquivos de estilo externo e global. [Veja mais...](#)

6. **`src/index.html`**

1. ..

7. Módulo raiz usados para roteamento de componentes:

1. **`./src/app/app.module.ts`**

1. As rotas informam ao roteador qual visualização exibir quando um usuário clica em um link ou cola uma URL na barra de endereço do navegador.
2. Este arquivo contém a classe com decorador `NgModule`.
3. Os componentes `providers` e `bootstrap` são declarados somente no módulo raiz.
4. Só tem sentido usar rotas se a aplicação tiver mais um componente.

2. **`./src/app/app-routing.module.ts`**

1. [Veja a vídeo aula sobre rotas](#)
2. Este arquivo é usado para declarar as rotas da aplicação. [Veja mais...](#)
3. As rotas são adicionadas na matriz:
`const routes: Routes = [];`

1. Cada rota nesta matriz é um **objeto JavaScript** que contém duas propriedades. A primeira propriedade `path`, define o caminho do URL para a rota. A

segunda propriedade **component**, define o componente que o Angular deve usar para o caminho correspondente.

2. Exemplo:

```
const routes: Routes
= [
  { path: 'first-
component', component:
FirstComponent },
  { path: 'second-
component', component:
SecondComponent },
  { path: 'nome-do-
component', component:
NomeDoComponent },
  ...
];
```

4. O arquivo **src/polyfills.ts** é usado para importar as bibliotecas auxiliares.

3. Referências

1. [Angular CLI](#)
2. [Angular CLI: Estrutura do projeto](#)
3. [Vídeo aula sobre criação de novo projeto com a cli](#)
4. [CLI ng new](#)
5. [arquivos de configurações](#)
6. [readme.md](#)
7. [Componentes de estilos = css](#)
8. [Sintaxe do modelo...](#)
9. [templates-and-views](#)
10. [Componentes Visão Geral](#)
11. [Testando componentes](#)
12. [Definindo uma rota básica](#)
13. [NgModule](#)
14. [providers](#)
15. [bootstrap](#)
16. [Vídeo aula sobre módulos](#)
17. [Veja a vídeo aula sobre rotas](#)
18. [Interface Route](#)
19. [Configuração de ativos.](#)
20. [Configuração de ambiente do projeto](#)

21. [Arquivos de estilo externo e global](#)3. 3. **Como criar um componente usando o angular/cli**

1. [Visão geral da CLI e referência de comando](#)
2. [Criando um componente usando a CLI Angular.](#)
3. **Exemplo de como criar componente no prompt do linux.**

```
# Cria componente de nome home
ng generate component pages/home
```

4. Referências:

1. [Visão geral da CLI e referência de comando](#)
2. [Criação de um componente usando a CLI Angular](#)
3. [Comando **ng generate component**.](#)

5. 4. **Como criar biblioteca com o Angular 2**

1. [Visão geral das bibliotecas angulares](#)
2. [Veja como criar bibliotecas](#)
3. O flag **--create-application=false** do comando **ng new** cria um novo projeto de aplicativo inicial na pasta '**src**' do novo espaço de trabalho. Quando falso, cria um espaço de trabalho vazio sem aplicativo inicial. Você pode então usar o comando generate application para que todos os aplicativos sejam criados na pasta de projetos. [Veja mais...](#)
4. [O como e por que usar o sinalizador --create-application com o Angular CLI para criar um espaço de trabalho sem o aplicativo](#)
5. **Exemplo de como criar biblioteca com o Angular 2.**

1. Como criar workspace vazio:

```
ng new my-workspace --create-application=false
```

2. Como criar uma biblioteca com angula cli:

```
cd my-workspace
ng generate library my-lib
```

6. Referências:

1. [Visão geral das bibliotecas angulares](#)

2. [Como criar um espaço de trabalho angular.](#)
3. [O como e por que usar o sinalizador --create-application com o Angular CLI para criar um espaço de trabalho sem o aplicativo](#)
4. [The Angular Library Series - Criando uma biblioteca com Angular CLI](#)
5. [O Projeto Biblioteca](#)
6. [foo \(em programação de software\)](#)

7. [ BACK]

5. Como gerar projeto para publicar na web

1. Quando executamos o comando **ng new nome-do-projeto** o mesmo adiciona no objeto **"scripts": {}** do arquivo **package.json** a propriedade **"build": "ng build",**. [Veja mais...](#)
2. Para gerar a pasta **./dist/nome-do-projeto** executar o seguinte comando:

```
# Executando via comando ng
ng build

# ou usar somente a abreviação de build = b
ng b

# ou pode ser gerado também usando o comando:
npm run build
```

3. Referências

1. [ng build](#)

4. [ BACK]

6. Como criar módulos

1. Ao criar uma aplicação com o comando **ng new nome-do-projeto --routing** o módulo é criado normalmente, porém quando a aplicação é muito grande precisamos modularizar e neste caso, o comando abaixo se faz necessário. [Veja a vídeo aula para mais informações.](#)
2. Criando módulo usando o angular cli.

```
ng g m NomeDoModulo
```

3. Referências:

1. [Vídeo aula sobre módulos](#)
2. [Definindo uma rota básica](#)
3. [NgModule](#)

4. [ BACK]

7. Como usar rotas em Angular 2

1. Arquivos para implementação de Rotas

1. O comando **ng new nome-do-projeto --routing** cria dois arquivos, sendo um (**src/app/app-routing.module.ts**) para registrar as rotas para os componentes e outro (**src/app/app.module.ts**) para registrar os componentes do projeto:

1. O arquivo **src/app/app-routing.module.ts** é usado para registrar as rotas para os componentes que estarão acessível a partir de uma requisição do browser .

1. Código typescript criado automaticamente.

```
import { NgModule } from
 '@angular/core';
import { RouterModule, Routes } from
 '@angular/router';

//Matriz Router criada automaticamente
paras as rotas
const routes: Routes = [ ];

/* Exemplo de rota para 3 componentes
const routes: Routes = [
  { path: 'first', component:
FirstComponent },
  { path: 'second', component:
SecondComponent },
  { path: 'rota', component:
NomeDoComponent },
  ...
*/
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
//Classe exportada para a classe AppModule
abaixo.
```

2.  [BACK]

2. O arquivo **src/app/app.module.ts** é usado para registrar todos os componentes (units do Delphi) do projeto.

1. Código typescript criado automaticamente.

```
import { NgModule } from
 '@angular/core';
import { BrowserModule } from
 '@angular/platform-browser';

import { AppRoutingModule } from './app-
routing.module';
import { AppComponent } from
 './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

2.  [BACK]

3. Com a criação dos dois arquivos acima ao abrir o browser continuaremos a ver a página padrão do Angular na tela sem visualizar rotas porque nem uma rota foi criada acima.

4.  [BACK]

2. Exibindo as páginas do projeto:

1. Adicionar componentes ao projeto:

1. Código shellscript:

```
# Página para login do usuário
ng generate c page/login

# Página de inscrição de usuário
ng generate c page/signup
```

2.  [BACK]

2. Adicionar rotas ao projeto:

1. No arquivo **app-routing-module.ts** adicione as seguintes linhas:

```
// Na secção de importação importe os dois
componentes abaixo
import { LoginComponent } from
'./pages/login/login.component';
import { SignupComponent } from
'./pages/signup/signup.component';

//Na matriz de Routes adicione os dois
objetos abaixo:
const routes: Routes = [
  {path: 'login', component: LoginComponent},
  {path: 'signup', component: SignupComponent},
];
```

1. [ BACK]

2. No arquivo **app-module.ts** adicione as seguintes linhas:

```
// Na secção de importação importe os dois
componentes abaixo
import { LoginComponent } from
'./pages/login/login.component';
import { SignupComponent } from
'./pages/signup/signup.component';

// Matriz declarations adicione os dois
componentes importados
declarations: [
  LoginComponent,
  SignupComponent,
],
```

1. [ BACK]

3. Notas:

1. Quando configuramos uma rota e navegamos até ela, o Angular recupera a URL, checa no arquivo de rotas e tenta carregar o componente na tela. Neste caso, o que acontece é que o Angular recuperou a URL <http://localhost:4200/login>, encontrou uma rota com este nome, localizou o componente que devia renderizar na tela, mas não conseguiu encontrar ONDE, na tela, ele deveria renderizar este componente. Vamos então para página **app.component.html**, remover todo conteúdo dela e deixar apenas as tags. Desta forma, ao navegar para URL

<http://localhost:4200/login>, conseguimos ver o texto "Login Works!", padrão dos componentes do Angular.

2. É importante notar que qualquer HTML contido na página **app.component.html** será exibido em todas as páginas do nosso App, então, é importante que nesta página seja declarado o menu de opções e a tag **<router-outlet></router-outlet>** nele.

3. 

3. 

3. Rotas filhas:

1. Veja vídeo aula: [Curso Angular #60: Rotas Filhas](#)
2. Veja vídeo aula: [Curso Angular #61: Rotas Filhas: desenvolvendo as telas](#)
3. 

4. Guardas de Rota ([Prevenir o acesso não autorizado](#)).

1. Para prevenir o acesso não autorizado é necessário a proteções de rota para evitar que os usuários naveguem para partes de um aplicativo sem autorização. Os seguintes protetores de rota estão disponíveis em Angular:

1. [CanActivate](#)

1. Veja o vídeo: [Curso Angular #64: Usando Guarda de Rotas: CanActivate](#)

2. [CanActivateChild](#)

1. Veja a vídeo aula: [Curso Angular #65: Usando Guarda de Rotas: CanActivateChild](#)

3. [CanDeactivate](#)

1. Veja a vídeo aula: [Curso Angular #66: Usando Guarda de Rotas: CanDeactivate](#)
2. Veja a vídeo aula: [Curso Angular #67: Usando Guarda de Rotas: CanDeactivate com Interface Genérica](#)

4. [Resolve](#)

1. Veja a vídeo aula: [Curso Angular #68: Guarda de Rotas: Resolve: carregando dados antes da rota ser ativada](#)

5. [CanLoad](#)

1. Veja a vídeo aula: [Curso Angular #69: Usando Guarda de Rotas: CanLoad: como não carregar o módulo sem permissão](#)

2. Para usar protetores de rota, considere o uso [de rotas sem componentes](#), pois isso facilita a proteção de rotas secundárias. Crie um serviço para o seu guarda:

1. Código ShellScript:

```
ng generate guard your-guard
```

3. ..

4. [ BACK]

5. Referências:

1. [@angular/roteador](#)
2. [angular rotas guardas navegacao](#)
3. [angular-rotas-guardas-navegacao](#)
4. [Curso Angular 2 #49: Rotas: Introdução](#)
5. [Curso Angular #50: Rotas: Configurando rotas simples](#)
6. [Curso Angular #51: Rotas: RouterLink: definindo rotas no template](#)
7. [Curso Angular #52: Rotas: Aplicando CSS em rotas ativas](#)
8. [Curso Angular #53: Rotas: Definindo e extraindo parâmetros de roteamento](#)
9. Guarda de rotas:
 1. [Curso Angular #64: Usando Guarda de Rotas: CanActivate](#)
 2. [Curso Angular #65: Usando Guarda de Rotas: CanActivateChild](#)
 3. [Curso Angular #66: Usando Guarda de Rotas: CanDeactivate](#)
 4. [Curso Angular #67: Usando Guarda de Rotas: CanDeactivate com Interface Genérica](#)
 5. [Curso Angular #68: Guarda de Rotas: Resolve: carregando dados antes da rota ser ativada](#)
 6. [Curso Angular #69: Usando Guarda de Rotas: CanLoad: como não carregar o módulo sem permissão](#)

10. .

6. [ BACK]

8. Assunto 08

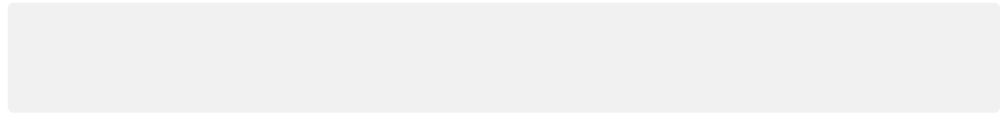
1. Descrição do conteúdo.
2. **Exemplo do assunto 08.**
 1. Descrição do exemplo

3. [ BACK]

9. Assunto 09

1. Descrição do conteúdo.
2. **Exemplo do assunto 09.**

3. Descrição do exemplo



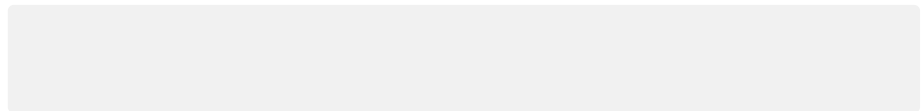
4. [← BACK]

10. Assunto 10

1. Descrição do conteúdo.

2. Exemplo do assunto 10.

1. Descrição do exemplo



3. [← BACK]

11. [← BACK]

4. REFERÊNCIAS GLOBAIS

1. [Site oficial para produzir este documento](#)
2. [Documentação de todas as APIs do angular 2](#)
3. [Como adicionar bootstrap a um projeto CLI Angular](#)
4. [Exemplos de bootstrap com angular 2](#)
5. [Projeto Angular Bootstrap com exemplos](#)
6. [Curso Angular #64: Usando Guarda de Rotas: CanActivate](#)
7. [ember-cli - Uma estrutura para desenvolvedores web ambiciosos.](#)

8. [← BACK]

5. HISTÓRICO

1. 22/06/2021

- ☒ Criar este documento baseado no o_que_e_angular.md ;
- ☒ Escrever tópico Objetivos;
- ☒ Escrever tópico Pre-requisitos
- ☒ Escrever tópico Benefícios
- ☒ Escrever tópico desvantagens

■ [← BACK]

2. 24/06/2021

- ☒ Escrever tópico Conteúdo estudado
- ☒ Escrever tópico Referências



3. 02/07/2021

- ☒ Escrever o exemplo: Como criar formulário de login no angular
- ☐ Escrever tópico Referências
- ☐ Atualizar o histórico deste documento.
- ☐ Testar este documento depois após uma semana de concluído.

